WEEK 2 · DAY 2 · OVERVIEW

Rolling Windows and Volatility



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

Two of our screens show the 10-year's volatility as 5.29 and 84.05 bp, from the same Treasury file, and neither says what it is. Give marketdata one definition: a window in rows, trailing, full windows only, the sample standard deviation of daily changes in bp, per day unless the name says annual. Tie it to Excel's STDEV.S filled down, prove no future row ever reaches today's number, and show me the 10-year over about one, three and twelve months as of 2 October. Describe how much it moved, with no view on what comes next.

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

Your Route Through Day 2

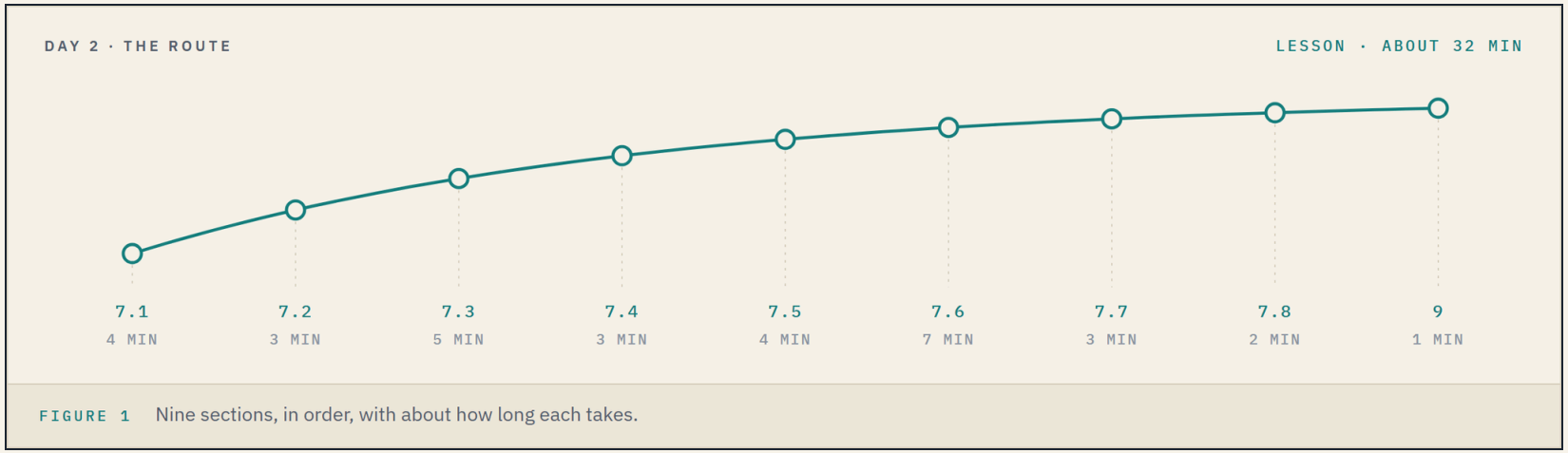


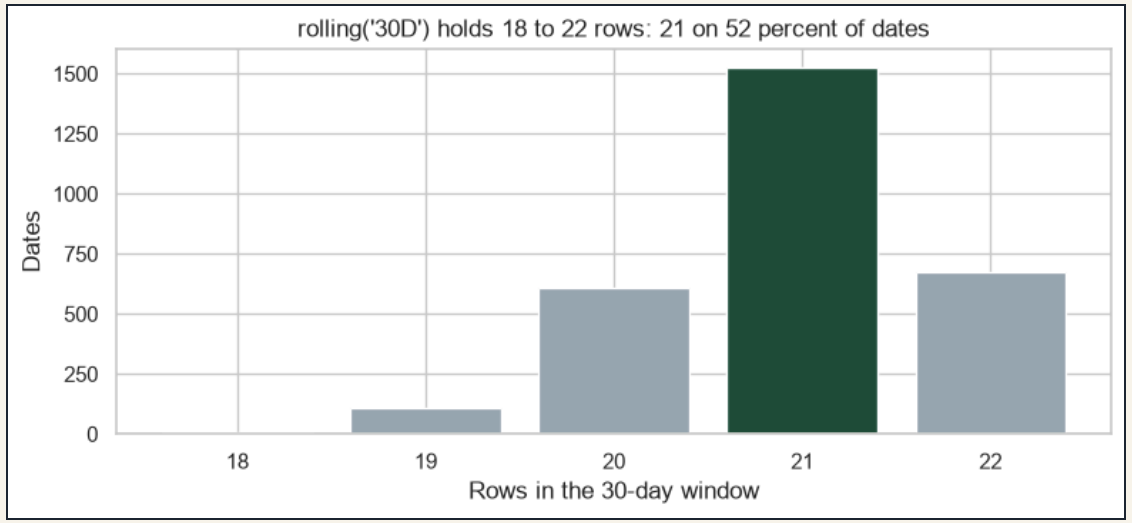
FIGURE 1 Nine sections, in order, with about how long each takes.

	SECTION	ABOUT
7.1	A Window in Rows Against a Window in Days	4 min
7.2	Full Windows Only: Trailing, Right-Aligned, `min_periods`	3 min
7.3	Volatility in bp: `rolling_vol_bp`	5 min
7.4	Sample Against Population	3 min
7.5	Annualising: The Factor, Labelled	4 min

	SECTION	ABOUT
7.6	Lookahead: `assert_no_lookahead`	7 min
7.7	Yesterday's File, and Back: `git restore --source`	3 min
7.8	21, 63 and 252 Rows on One Chart	2 min
9	Bring It Home	1 min

Time. About 32 minutes for the lesson, then about 12 for the four exercises.

7.1 A Window in Rows Against a Window in Days



COUNT ROWS IN 30 DAYS

```
days_30 = ten_changes.rolling('30D').count()
counts = days_30.loc['2015-02':].value_counts().sort_index()
share = counts[21] / counts.sum() * 100
```

EXCEL SIDE BY SIDE

JOB	EXCEL
Rows in the last 30 calendar days, filled down	`=COUNTIFS($A$3:$A$2941,">"&A3-30,$A$3:$A$2941,"<="&A3)`

A window in rows always holds the same number of changes, a window in days does not. The course counts rows: 21, 63 and 252, "about one month, three months, one year", conventions rather than calendar months.

7.2 Full Windows Only: Trailing, Right-Aligned, min_periods

The range is the window: =AVERAGE(B2:B4) in C4, filled down

	B: change	C: 3-row mean	
2	1	empty: not full	
3	3	empty: not full	
4	2	=AVERAGE(B2:B4)	2
5	5	=AVERAGE(B3:B5)	3.333
6	4	=AVERAGE(B4:B6)	3.667
7	6	=AVERAGE(B5:B7)	5

```
=AVERAGE(OFFSET(B4,-2,0,3,1))
```

B4	start from today's row
-2	move 2 rows up
0	and 0 columns across
3	take 3 rows
1	and 1 column

the same 3 rows, B2:B4, so the same 2

Each row's formula covers the 3 rows ending on it: the window slides down one row at a time.

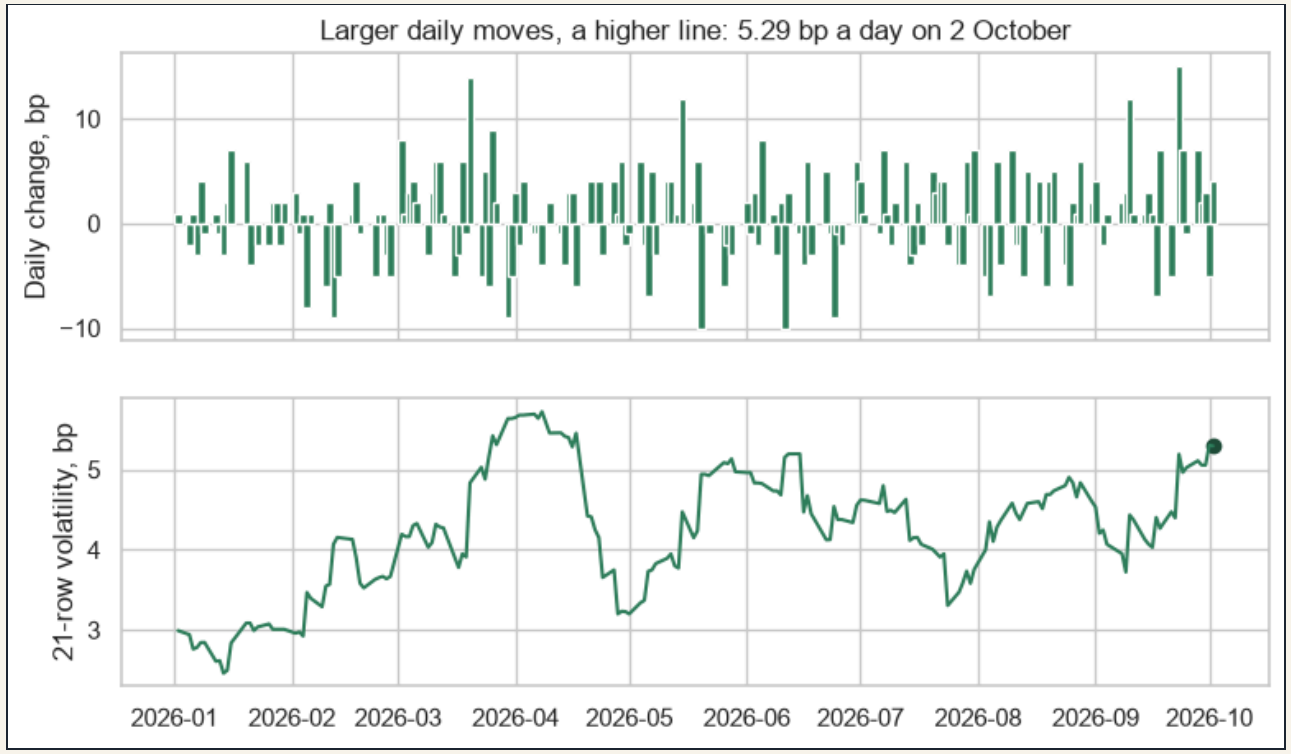
FIGURE 3 Six changes in B2 to B7: C4 holds =AVERAGE(B2:B4), 2, and C2 and C3 stay empty until a full window.

EXCEL SIDE BY SIDE

JOB	EXCEL
3-row mean, C4 filled down	`=AVERAGE(B2:B4)`
The same window by offset	`=AVERAGE(OFFSET(B4,-2,0,3,1))`
A growing range, the changes in A1 to A3	`=AVERAGE(A$1:A1)` filled down

Full windows only: `min_periods` equals the window, so the first window - 1 values are NaN, as Excel leaves the cells above the first full range empty.

7.3 Volatility in bp: rolling_vol_bp



THE NOTE'S 5.29

```
vol21 = md.rolling_vol_bp(ten_changes, window=21)
print(f'2026-10-02: {vol21.iloc[-1]:.2f} bp a day')
```

```
2026-10-02: 5.29 bp a day
```

EXCEL SIDE BY SIDE

JOB	EXCEL
21-row volatility, G23 filled down	`=STDEV.S(F3:F23)`
The same by offset, N23	`=STDEV.S(OFFSET(F23,-20,0,21,1))`

`rolling_vol_bp` **is Excel's** `STDEV.S` **filled down, on every row of 2026: 5.29 bp a day for the 10 Yr on 2 October.**

Choose between the sample and population figure.

7.4 Sample Against Population

TWO STANDARD DEVIATIONS

	DIVIDES BY	PANDAS	NUMPY	EXCEL
Sample	n - 1	`.std()`, `ddof=1` the default	`np.std(x, ddof=1)`	`STDEV.S`
Population	n	`.std(ddof=0)`	`np.std(x)`, `ddof=0` the default	`STDEV.P`

THE LAST 21 CHANGES

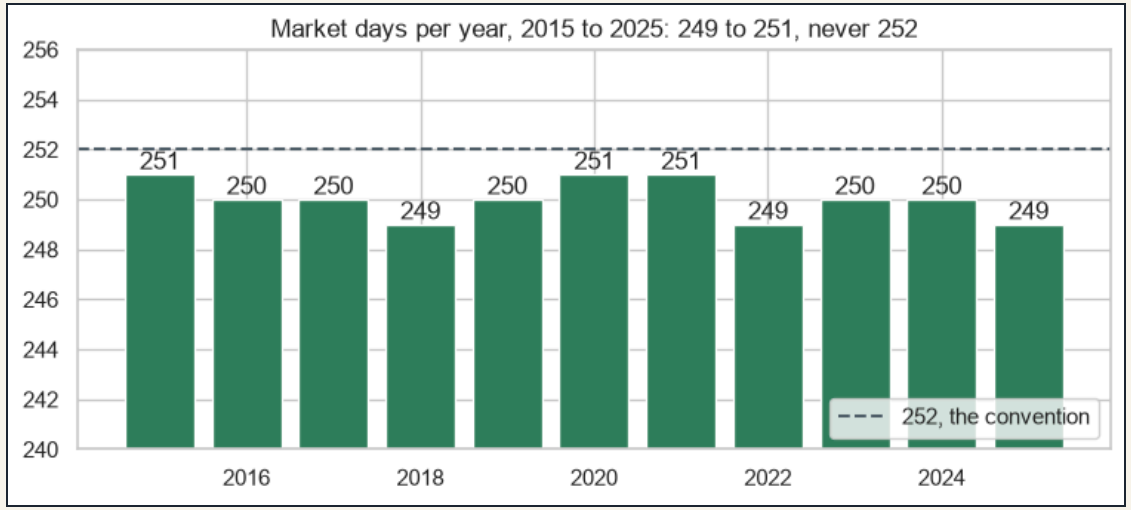
```
last_21 = ten_changes.iloc[-21:]
sample, population = last_21.std(ddof=1), last_21.std(ddof=0)
print(f'sample {sample:.6f}, population {population:.6f}, '
      f'np.std {np.std(last_21.to_numpy()):.6f} bp')
print(f'sample / population: {sample / population:.6f}')
```

```
sample 5.294651, population 5.167051, np.std 5.167051 bp
sample / population: 1.024695
```

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL	RESULT
Population volatility, H23 filled down	`.rolling(21).std(ddof=0)`	`=STDEV.P(F3:F23)`	Within 2.4e-14 bp, the last 5.167050676973533
One value	a window of 1	`=STDEV.S(A1)`	`#DIV/0!`, which is why the window is at least 2

7.5 Annualising: The Factor, Labelled



THE NOTE'S 84.05

```
vol21_annual = md.rolling_vol_bp(ten_changes, window=21, annualise=True)

84.05 bp a year = 5.29 x 15.8745
```

EXCEL SIDE BY SIDE

JOB	EXCEL
Rows in 2015	`=COUNTIFS($A$2:$A$2941,">="&DATE(2015,1,1),$A$2:$A$2941,"<"&DATE(2016,1,1))`

84.05 is 5.29 times the square root of 252, one measure in two units. The factor is a convention, so it lives in the name (_annual) and the argument, never in silence.

7.6 Lookahead: assert_no_lookahead

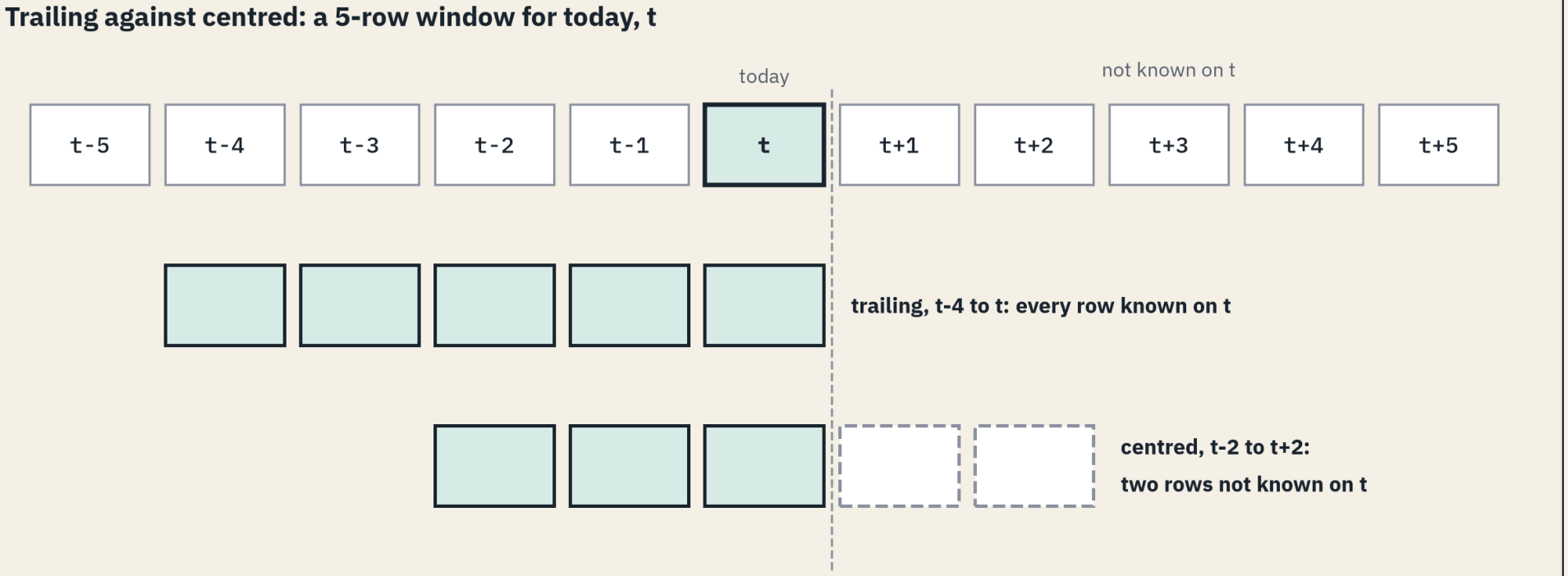


FIGURE 4 A trailing 5-row window ends on t and is known on t, while a centred one needs two rows after t.

A centred window reaches past t.

The Check That Catches It

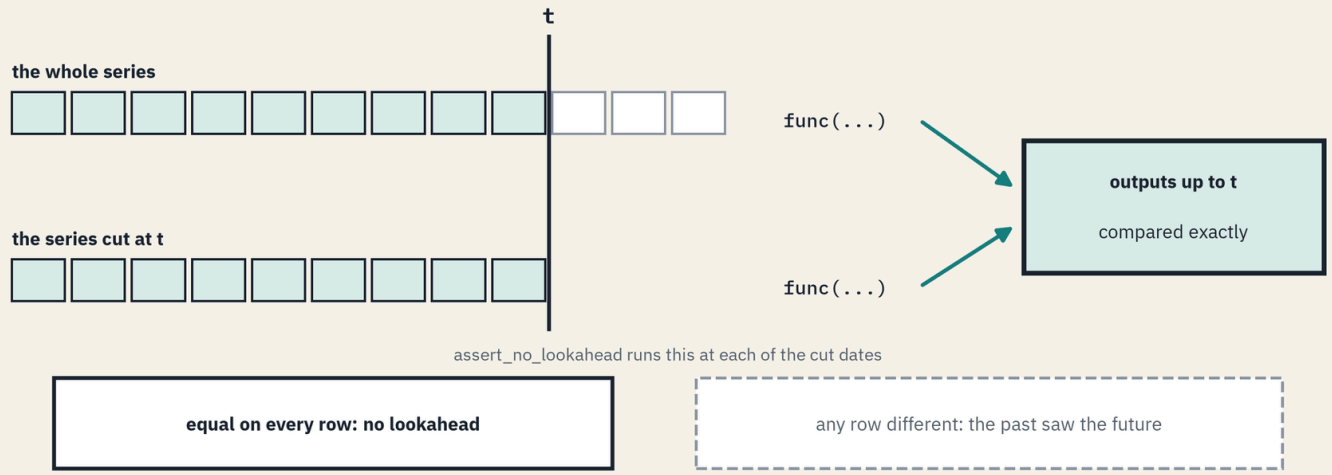


FIGURE 5 The same function runs on the whole series and on the series cut at t, and the outputs up to t must match.

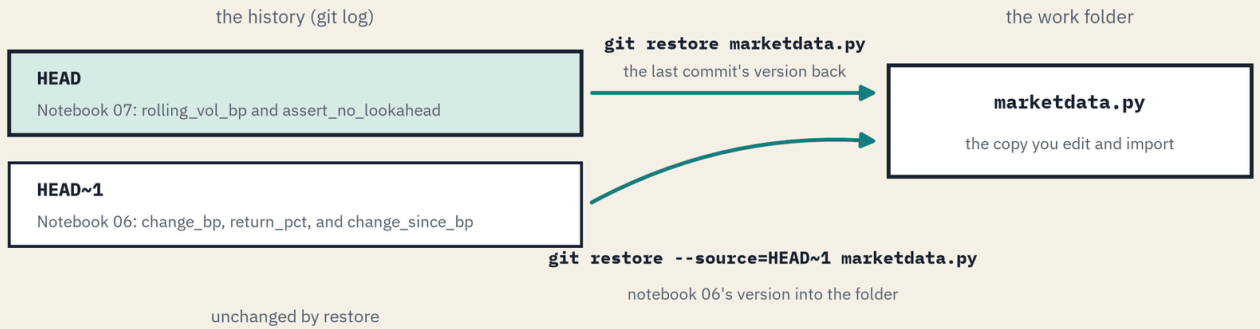
THE HELPER

```
def assert_no_lookahead(func, series, cut_dates):
    full = func(series)
    for cut in cut_dates:
        cut_result = func(series.loc[:cut])
```

The rule: a function's output up to t is the same whether the series stops at t or runs on, compared exactly.

7.7 Yesterday's File, and Back: git restore --source

The folder borrows a version: the commits stay where they are



restore copies a file from a commit into the folder; it never moves HEAD or makes a commit.

FIGURE 6 git restore --source=HEAD~1 copies notebook 06's marketdata.py into the folder, and a plain restore brings today's back.

COMMIT THE DAY'S WORK

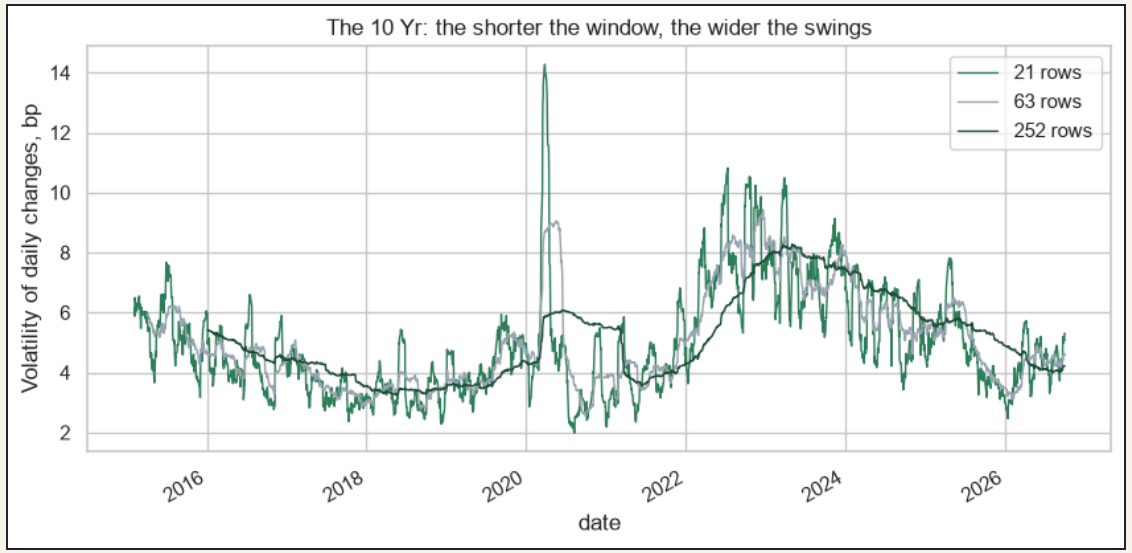
```
!git -C "{work}" diff --stat
!git -C "{work}" log --oneline -3
```

BACK TO TODAY'S

```
!git -C "{work}" restore marketdata.py
!git -C "{work}" status --short
```

Plain `restore` throws away every change since the last commit, so read `status` first, as Course 2 warned.

7.8 21, 63 and 252 Rows on One Chart



EXCEL SIDE BY SIDE

JOB	EXCEL
63 rows, J65 filled down	`=STDEV.S(F3:F65)`
252 rows, K254 filled down	`=STDEV.S(F3:F254)`
The 21-row high	`=MAX(G23:G2941)`

On 2 October 2026 the 10 Yr's daily changes had a volatility of 5.29 bp over 21 rows, 4.63 over 63 and 4.22 over 252. The note is answered.

Copy the files home and commit them.

Bring It Home

IN A POWERSHELL TERMINAL

```
# the work folder: use the name your setup cell printed
$work = "$env:TEMP\pfi_course3_07_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 07: rolling_vol_bp and assert_no_lookahead"
git restore --source=HEAD~1 marketdata.py
git status --short
git restore marketdata.py
```

You Can Now

- Tell a window in rows from a window in calendar days, and say why the course counts rows.
- Keep to full windows with **`min_periods`**, as Excel leaves the cells above the first full range empty.
- Measure volatility with **`rolling_vol_bp`**, tied to Excel's **`STDEV.S`** filled down.
- Choose between the sample and population standard deviation, and name **`ddof`**.
- Annualise with the square root of 252, and label it.
- Prove a window function uses no future data with **`assert_no_lookahead`**.
- Put yesterday's file in the folder with **`git restore --source=HEAD~1`**, and bring today's back.

NEXT
WEEK 2 · DAY 3

Z-Scores and Percentile Ranks

Where today's level sits in its own last year: the z-score and the percentile rank over a trailing 252-row window, tied to Excel, tested for lookahead, and described without a forecast.

